

Optimizing Shipper Assignment and Routing in Dynamic Delivery Systems

1st Ha Xuan Son
UET - VNU
23021681@vnu.edu.vn

2nd Nguyen Phan Dang
UET - VNU
23021527@vnu.edu.vn

3rd Van Le Quoc Anh
UET - VNU
23021472@vnu.edu.vn

Abstract—This report presents the optimization problem of multi-agent delivery in a dynamic environment, where multiple shippers operate simultaneously on a grid map to pick up and deliver orders that appear in real time. Each order contains a pickup location, delivery location, weight, priority level, and deadline, while the system may also include surge periods and order-concentrated hotspot areas. The objective is to maximize the total reward by balancing on-time delivery, high-priority order handling, movement cost reduction, and conflict avoidance among shippers.

To address this problem, our team designs and evaluates coordination strategies based on shortest-path search, heuristic order selection, adaptive task assignment, and local collision handling. Algorithms such as Greedy-BFS and MAPD-CBS are applied to combine computational efficiency with solution quality in an online setting. Experimental results show that combining effective order assignment, deadline-aware scoring, hotspot detection, and path planning can improve delivery performance across different map configurations and order densities.

Index Terms—Multi-agent systems, dynamic delivery optimization, task assignment, path planning, vehicle routing problem, Greedy-BFS, Conflict-Based Search, MAPD-CBS, real-time scheduling, grid-based routing, deadline-aware delivery, hotspot detection.

I. INTRODUCTION

With the rapid growth of e-commerce and on-demand delivery services, coordinating multiple shippers within the same delivery system has become an important and practical optimization problem. Orders do not necessarily appear at the beginning of the simulation; instead, they may be generated continuously over time, with different pickup locations, delivery locations, weights, priority levels, and deadlines. During surge periods, orders may also concentrate around specific hotspot areas, creating local bottlenecks and increasing the risk of late deliveries or inefficient movement.

This report studies the multi-agent delivery problem in a dynamic environment. A team of shippers moves on a grid map with obstacles and performs movement, pickup, and delivery actions at each time step. The main objective is to maximize the total reward, which depends on on-time delivery, order priority, package weight, and movement cost. Since order information is revealed in real time, the algorithm must make online decisions based on the current system state rather than relying on complete future information.

To solve this problem, our approach combines task assignment and path planning. Strategies such as Greedy-BFS are used to select suitable orders and generate efficient movements

based on shortest-path distance, deadlines, and priority levels. In addition, the MAPD-CBS approach is explored to improve multi-agent coordination and reduce path conflicts in more complex scenarios. By integrating adaptive heuristics, workload evaluation, and hotspot detection, the solver can respond more effectively to changes in the environment.

This report presents the problem formulation, proposed algorithms, solver implementation, and experimental results under different configurations. Through these experiments, we evaluate the effectiveness of each strategy in optimizing the final score, reducing movement cost, and improving on-time delivery performance in a real-time multi-agent delivery system.

II. PROBLEM FORMULATION

A. Environment Model

The delivery environment is represented as an $N \times N$ grid map. Each cell in the grid can either be free space or an obstacle. Shippers are only allowed to move through valid cells and cannot pass through obstacles or leave the map boundary. At each time step, the environment state includes the current positions of all shippers, the revealed orders, newly generated orders, completed orders, and the current simulation time.

The system runs for T time steps. Orders do not necessarily appear at the beginning of the simulation; instead, they are generated dynamically over time. During surge periods, the order density may increase and concentrate around hotspot areas, creating local workload imbalance in the system.

B. Agent and Order Model

The system consists of C shippers operating simultaneously on the grid map. Each shipper i is described by its current position (x_i, y_i) , maximum carrying weight $W_{\max}(i)$, maximum package capacity $K(i)$, and the list of orders currently being carried. Initially, no two shippers occupy the same cell.

Each order g_j is represented by:

$$g_j = \langle sx_j, sy_j, ex_j, ey_j, et_j, w_j, p_j \rangle$$

where (sx_j, sy_j) denotes the pickup location, (ex_j, ey_j) denotes the delivery location, et_j is the delivery deadline, w_j is the package weight, and p_j is the priority level. A higher priority value indicates a more important order that should be handled earlier.

C. Action Space

At each time step, each shipper performs an action consisting of two components: a movement action and a cargo operation. The movement action is selected from:

$$move \in \{S, L, R, U, D\}$$

where S means staying in place, L means moving left, R means moving right, U means moving up, and D means moving down. The cargo operation includes three types: doing nothing, picking up an order at the current cell, or delivering a carried order if the shipper is at the correct delivery location.

A general action is defined as:

$$a_i^t = (move_i^t, cargo_op_i^t)$$

Within each time step, movement is executed first, followed by pickup and then delivery.

D. Reward Function

The reward function is designed to evaluate the delivery performance of the whole system. When an order is successfully delivered, the system receives a reward depending on the order priority, package weight, and delivery time relative to its deadline. High-priority or on-time orders usually yield higher rewards, while late deliveries may reduce the reward or introduce penalties.

In addition to delivery rewards, the system also considers the movement cost of shippers. Each movement step may decrease the total score, so the algorithm must balance accepting more orders, delivering them on time, and avoiding unnecessary travel.

The reward at time step t can be generally expressed as:

$$R_t = \sum_{g_j \in D_t} reward(g_j, t) - \lambda \sum_{i=1}^C cost(move_i^t)$$

where D_t is the set of orders delivered at time t , $reward(g_j, t)$ is the reward obtained from delivering order g_j , $cost(move_i^t)$ is the movement cost of shipper i , and λ is a balancing coefficient for movement cost.

E. Objective Function

The objective of the problem is to find a coordination policy for the entire shipper team that maximizes the accumulated reward over T time steps. This policy must decide which shipper should handle which order, which direction each shipper should move, when to pick up packages, and when to deliver them, based on the observed state at each time step.

The objective function is formulated as:

$$\max_{\pi} \sum_{t=0}^T R_t$$

where π denotes the coordination policy of the system and R_t is the reward obtained at time step t . Since orders appear dynamically and future information is not known in advance, the problem is online and requires the algorithm to make efficient decisions within a limited time.

III. BENCHMARK CONFIGURATION AND EVALUATION METRICS

A. Phase 1 Configurations

In Phase 1, the algorithms are evaluated on six public benchmark configurations, denoted from C1 to C6. These configurations gradually increase in difficulty in terms of grid size, number of shippers, number of orders, and simulation horizon. Each map is represented as an $N \times N$ grid, where obstacles create narrow passages or bottleneck areas that affect shipper movement.

Config	N	C	G	T
C1	7	2	15	240
C2	10	2	25	240
C3	12	3	40	360
C4	15	4	60	600
C5	18	5	80	780
C6	20	5	100	960

TABLE I: Phase 1 benchmark configurations

Here, N denotes the grid size, C is the number of shippers, G is the total number of orders, and T is the simulation horizon. Smaller configurations such as C1 and C2 are used to verify the basic correctness of the algorithm, while larger configurations such as C5 and C6 evaluate scalability, congestion handling, and multi-shipper coordination in more complex environments.

B. Metrics

The primary metric used to compare algorithms is the *net reward*, which is the total delivery reward after subtracting movement cost. This value directly reflects the overall effectiveness of the algorithm in selecting orders, delivering them on time, and avoiding unnecessary movement. For each configuration, the main score is computed as:

$$Score = NetReward = TotalReward - TotalMoveCost$$

In addition to the main score, several auxiliary metrics are used to analyze the behavior of the algorithm:

$$DeliveryRate = \frac{Delivered}{TotalOrders}$$

$$OnTimeRate = \frac{OnTime}{Delivered}$$

$$Missed = TotalOrders - Delivered$$

where *Delivered* is the number of successfully delivered orders, *TotalOrders* is the total number of orders in the configuration, *OnTime* is the number of orders delivered before or at their deadlines, and *Missed* is the number of undelivered orders. The number of late deliveries and the runtime are also recorded to evaluate the stability and computational efficiency of the algorithm.

The total score of a method over the entire Phase 1 benchmark is computed as the sum of *net reward* values across all configurations:

$$TotalScore = \sum_{k=1}^6 NetReward_k$$

where $NetReward_k$ is the net reward obtained on configuration C_k . Therefore, a strong algorithm should not only achieve high scores on individual configurations but also maintain stable performance as the problem scale increases.

IV. PROPOSED METHODS

A. Greedy BFS

Greedy BFS is an online heuristic solver for the Multi-Agent Package Delivery problem. It combines shortest-path planning with Breadth-First Search (BFS) and greedy decision rules for selecting actions at each time step. Since the environment only reveals orders that have already appeared, the solver cannot access future orders and must continuously make decisions based on the current observation.

The main objective of Greedy BFS is to maximize the *net reward*, which is the total delivery reward after subtracting movement cost. To achieve this objective, the solver does not simply choose the nearest order. Instead, it jointly considers deadline, priority, expected reward, remaining package slots, remaining carrying capacity, shipper position, and possible local congestion on the map.

1) *Overall Pipeline*: At each timestep, Greedy BFS performs a short-horizon decision cycle consisting of the following steps:

- 1) Receive the current observation from the environment.
- 2) Update the hotspot tracker using newly revealed orders.
- 3) Select the delivery target for each shipper carrying packages.
- 4) Generate feasible pickup candidates for shippers with remaining capacity.
- 5) Choose either local policy or global policy depending on current workload.
- 6) Generate movement actions using BFS.
- 7) Reserve next cells to reduce direct collisions among shippers.
- 8) If a shipper is idle, reposition it toward a hotspot or regional anchor.
- 9) Submit actions to the environment and repeat at the next timestep.

The overall pipeline can be represented as:

Obs $\rightarrow$ Hotspot $\rightarrow$ Delivery
 $\rightarrow$ Pickup $\rightarrow$ BFS $\rightarrow$ Collision $\rightarrow$ Step

2) *BFS Path Planning and Cache*: Since the map is an unweighted grid and each movement step has the same cost, BFS is used to compute the shortest path between two valid cells. Given a start position s and a goal position g , BFS returns the shortest distance $dist(s, g)$ and the first move along that shortest path.

In the implementation, the solver maintains two caches:

$$DistanceCache[(s, g)] = dist(s, g)$$

$$NextMoveCache[(s, g)] = first_move(s, g)$$

These caches prevent repeated BFS computations for the same pair of positions. This is important because, at each

timestep, the solver may need to evaluate many shipper–order pairs. Without caching, pathfinding cost would grow quickly as the number of shippers and active orders increases.

If no valid path exists between two cells, the distance is assigned a large value INF , and the corresponding candidate is removed from consideration.

3) *Delivery Target Selection*: When a shipper is carrying one or more packages, the solver must decide which package should be delivered first. Greedy BFS uses a sorting key called `delivery_key`, which considers on-time feasibility, deadline slack, distance to dropoff, priority, and order id.

For shipper i and order j , the estimated completion time is:

$$finish_t = t + dist(pos_i, dropoff_j)$$

$$slack_j = et_j - finish_t$$

The delivery key is defined as:

$$f_{ij} = t + d(pos_i, dropoff_j), \quad (1)$$

$$l_{ij} = \begin{cases} 1, & \text{if } f_{ij} > et_j, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

$$K_{ij}^{del} = (l_{ij}, et_j - f_{ij}, d(pos_i, dropoff_j), -p_j, id_j). \quad (3)$$

Since smaller tuples are preferred, the solver first selects orders that can be delivered on time, then prioritizes tighter deadlines, shorter delivery distance, higher priority, and smaller order id for deterministic tie-breaking. This reduces the risk of holding packages for too long and frees capacity for future orders.

4) *Pickup Candidate Scoring*: For each feasible shipper–order pair, the solver estimates the total travel distance from the shipper’s current position to the pickup location and then from pickup to dropoff:

$$route_dist_{ij} = dist(pos_i, pickup_j) + dist(pickup_j, dropoff_j)$$

$$finish_t = t + route_dist_{ij}$$

$$slack_j = et_j - finish_t$$

The expected reward is computed using the same reward function as the environment:

$$ExpectedReward_j = delivery_reward(g_j, finish_t, T)$$

The value of an order per movement step is then defined as:

$$value_rate_{ij} = \frac{ExpectedReward_j}{route_dist_{ij} + 1}$$

In global mode, pickup candidates are ranked using:

$$q_{ij} = d(pos_i, pickup_j) + d(pickup_j, dropoff_j), \quad (4)$$

$$v_{ij} = \frac{R_j(t + q_{ij})}{q_{ij} + 1}, \quad (5)$$

$$K_{ij}^{\text{pick}} = (l_{ij}, -v_{ij} \cdot h_j, r_{ij}^{\text{reg}}, s_j, -p_j, q_{ij}, id_j). \quad (6)$$

This key prioritizes orders that can be delivered on time, provide high reward per movement step, have low regional penalty, reasonable deadline slack, high priority, and short route distance. Compared with nearest-order greedy selection, this scoring is more comprehensive because it considers not only pickup distance but also the actual value of completing the order.

5) *Local Policy and Global Matching*: Greedy BFS uses two assignment modes depending on current workload.

When the workload is light, the solver uses a *local policy*. Each shipper selects the most suitable order based on its current position. In this mode, nearby orders are often preferred because the system is not under heavy pressure and the overhead of global matching is unnecessary.

When the workload is high, the solver uses *global greedy matching*. It generates all feasible shipper–order pairs, scores them using `pickup_key`, sorts the list, and greedily assigns targets such that each shipper receives at most one new pickup target and each order is pursued by at most one shipper.

The switch between local and global modes is based on the current pressure:

$$pressure = active_orders + carried_orders$$

If *pressure* is small relative to the number of shippers, local policy is used. Otherwise, global matching is activated to reduce duplicate pursuit of the same order and to distribute orders more effectively across the team.

6) *Region-Aware Dispatch*: On large maps or maps with many obstacles, allowing a shipper to travel across the entire map for a low-value order can reduce overall efficiency. Therefore, the solver adds region-aware dispatch as a soft penalty mechanism.

When region policy is enabled, the map is divided into four regions:

TopLeft, TopRight, BottomLeft, BottomRight

If the pickup or dropoff location of an order lies outside the shipper’s current region, the order receives a `region_penalty`. A pickup in a different region is penalized more heavily than a dropoff in a different region, because traveling far to reach pickup is often the main reason why a shipper misses closer opportunities.

The region penalty does not forbid cross-region assignments. If an order has high reward, favorable deadline, or high priority, it can still be selected. Thus, the mechanism acts as a soft routing bias that reduces unnecessary long-distance travel without removing flexibility.

7) *Online Hotspot Detection*: The solver does not directly read hidden hotspot or surge information from the configuration file. Instead, hotspots are inferred online from newly revealed orders in the observation. Whenever a new order appears, its pickup location is added to the `HotspotTracker`.

The tracker uses a recent time window to identify areas with high order density.

For an order j , the hotspot bonus is derived from how close its pickup location is to recent order clusters:

$$hotspot_boost = 1 + 0.12 \cdot hotspot_score_j$$

This bonus is intentionally kept small so that it does not dominate reward and deadline information. As a result, the solver can exploit high-demand areas while still avoiding overcommitment to hotspots when current orders elsewhere are more valuable.

8) *Pickup Before Delivery*: An important improvement of Greedy BFS is that it allows a shipper to pick up an additional order before delivering a currently carried package, if doing so does not significantly harm the current delivery deadline. This mechanism helps the solver batch orders when the pickup point lies near the path to the current dropoff.

Assume that the shipper is carrying order d and is considering picking up order p . The direct delivery completion time is:

$$direct_finish = t + dist(pos_i, dropoff_d)$$

while the completion time after detouring through pickup is:

$$detour_finish = t + dist(pos_i, pickup_p) + dist(pickup_p, dropoff_d)$$

Pickup before delivery is accepted only if:

- the shipper still has remaining package slots and carrying capacity;
- the detour does not make an on-time carried order become late;
- the additional detour stays within an adaptive limit;
- the new pickup order has sufficiently high priority or value.

The detour limit is adjusted according to map size and current delivery distance:

$$adaptive_limit = \max \left(3, \min \left(\frac{N}{3}, \frac{dist(pos_i, dropoff_d)}{2} \right) \right)$$

This mechanism improves batching opportunities while limiting the risk of holding packages for too long or delaying currently carried orders.

9) *Distance Pruning*: To reduce computational cost and avoid inefficient decisions, the solver uses distance pruning on large maps or maps with bottleneck characteristics. An order is removed from the candidate list if:

- the shipper does not have enough package slots or carrying capacity;
- no valid path exists to pickup or from pickup to dropoff;
- the order has low priority but its pickup is too far away;
- the order is expected to be late and has low value per movement step;
- pursuing the order would likely cause the shipper to miss better nearby opportunities.

Pruning is enabled only when necessary to avoid over-pruning on small maps. This keeps the solver flexible on easier configurations while reducing noise on larger ones.

10) *One-Step Collision Avoidance*: The environment may cause shippers to lose movement opportunities when multiple shippers attempt to move into the same cell. To reduce direct collisions, Greedy BFS uses a `reserved_next_positions` mechanism. Once a shipper selects its next cell, that cell is reserved. Later shippers avoid selecting the same cell whenever possible.

If the best BFS move leads to an already reserved cell, the solver tries alternative moves:

$$S, U, D, L, R$$

and selects the valid move that results in the shortest remaining distance to the goal.

This is not full multi-agent path planning like CBS, but it is lightweight and effective for reducing one-step collisions in bottleneck areas.

11) *Idle Reposition*: When a shipper has no delivery target and no suitable pickup target, the solver does not always keep it stationary. On medium and large maps, an idle shipper may be repositioned toward the nearest hotspot or a regional anchor.

If online hotspots are available, the shipper moves toward the nearest hotspot:

$$goal_i = \arg \min_{h \in \text{Hotspots}} dist(pos_i, h)$$

If no hotspot is available and workload is low, the solver moves the shipper toward the anchor of its current region. The approximate anchor positions are:

$$\begin{aligned} TopLeft &= \left(\frac{N}{4}, \frac{N}{4} \right) \\ TopRight &= \left(\frac{N}{4}, \frac{3N}{4} \right) \\ BottomLeft &= \left(\frac{3N}{4}, \frac{N}{4} \right) \\ BottomRight &= \left(\frac{3N}{4}, \frac{3N}{4} \right) \end{aligned}$$

If an anchor lies on an obstacle, the solver selects the nearest valid free cell. This mechanism prevents idle shippers from staying in poor positions when future orders are sparse or spatially concentrated.

12) *Complexity Analysis*: Let C be the number of shippers, A the number of active orders at the current timestep, F the number of free cells on the map, and K the number of candidates retained after pruning. A single BFS call has complexity:

$$O(F)$$

If all shipper–order pairs are evaluated without caching, the worst-case cost per timestep is:

$$O(C \cdot A \cdot F)$$

With caching and distance pruning, the actual number of BFS calls is significantly reduced. If pruning retains K important candidates, the cost can be approximated as:

$$O(C \cdot A) + O(C \cdot K \cdot F)$$

In practice, many distance queries repeat across timesteps, so caching makes the solver much faster than the theoretical worst-case bound.

13) *Strengths and Limitations*: The main strengths of Greedy BFS are simplicity, determinism, interpretability, and good runtime performance. The solver does not depend on external libraries, making it suitable for offline evaluation environments. It also remains stable across different map configurations. By combining BFS caching, deadline-aware scoring, hotspot detection, region penalty, and one-step collision avoidance, the method significantly improves over a nearest-order greedy baseline.

However, Greedy BFS is still a short-horizon heuristic. It does not globally optimize multi-step routes, cannot accurately predict future orders, and only handles collisions at the one-step level. Therefore, on large configurations or maps with many bottlenecks, it can be less effective than methods with stronger multi-agent planning, such as MAPD-CBS.

B. VRP + OR-Tools

VRP + OR-Tools is implemented as an online solver for the Multi-Agent Package Delivery problem. Although the method name is kept as *VRP-OrTools* to match the expected solver interface, the actual implementation does not directly depend on the OR-Tools library. Instead, it uses a pure Python VRP-style heuristic, making it suitable for Kaggle or offline environments where external libraries may not be available.

The main idea is to approximate Vehicle Routing Problem behavior in an online environment. At each timestep, the solver observes the orders that have already appeared, evaluates all feasible shipper–order pairs, assigns orders using global greedy assignment, and then uses BFS to generate the next movement step. Since future orders are unknown, the solver does not construct a fixed route for the entire simulation. Instead, it continuously replans after every timestep.

1) *Design Objective*: The objective of the VRP-style solver is to maximize *net reward* by balancing three factors: order value, on-time delivery feasibility, and movement cost. Unlike a simple greedy method that only selects the nearest order, this method evaluates an order using the full route from the current shipper position to the pickup location and then from pickup to dropoff.

For each shipper i and order j , the solver considers:

- distance from the shipper to the pickup location;
- distance from pickup to dropoff;
- deadline and deadline slack;
- expected reward at the estimated delivery time;
- order priority;
- remaining package slots and carrying capacity;
- the shipper’s current region on the map.

This design makes assignment more stable than local per-shipper selection, especially when there are many active orders or when multiple shippers may pursue the same order.

2) *Overall Pipeline*: At each timestep, the VRP-style solver performs the following steps:

- 1) Receive the current observation from the environment.

- 2) Select delivery targets for shippers carrying packages.
- 3) Generate all feasible shipper–order candidate pairs.
- 4) Score each pair based on reward, deadline, distance, and capacity constraints.
- 5) Apply global greedy assignment to select pickup targets.
- 6) Generate movement actions using BFS next moves.
- 7) Reserve next cells to reduce direct collisions.
- 8) Submit actions to the environment and repeat until the simulation ends.

The overall pipeline can be represented as:

Obs $\rightarrow$ Delivery $\rightarrow$ Candidate
 $\rightarrow$ Assignment $\rightarrow$ BFS
 $\rightarrow$ Collision $\rightarrow$ Execute.

3) *BFS Routing and Caching*: The solver uses BFS to compute shortest paths on the obstacle grid. Since each movement step has the same cost, BFS is appropriate for finding the true shortest path between two cells, instead of relying only on Manhattan distance.

For two positions s and g , the solver stores:

$$\text{DistanceCache}[(s, g)] = \text{dist}(s, g)$$

$$\text{NextMoveCache}[(s, g)] = \text{first_move}(s, g)$$

Here, $\text{dist}(s, g)$ is the length of the shortest valid path, and $\text{first_move}(s, g)$ is the first movement action along that path. Caching reduces repeated BFS computations because the same position pairs can be queried multiple times during order scoring and action generation.

If no valid path exists between two cells, the distance is set to INF , and the corresponding shipper–order pair is removed from the candidate list.

4) *Delivery Target Selection*: For a shipper carrying packages, the solver chooses the delivery target using `delivery_key`. For shipper i and order j , the estimated completion time is:

$$\text{finish_t} = t + \text{dist}(\text{pos}_i, \text{dropoff}_j)$$

$$\text{slack}_j = \text{et}_j - \text{finish_t}$$

The delivery key is:

$$f_{ij}^{\text{del}} = t + d(x_i, e_j), \quad (7)$$

$$l_{ij}^{\text{del}} = \begin{cases} 1, & \text{if } f_{ij}^{\text{del}} > \text{et}_j, \\ 0, & \text{otherwise.} \end{cases} \quad (8)$$

$$K_{ij}^{\text{del}} = (l_{ij}^{\text{del}}, \text{et}_j - f_{ij}^{\text{del}}, d(x_i, e_j), -p_j, id_j). \quad (9)$$

where:

$$\text{late_flag} = \begin{cases} 1, & \text{if } \text{finish_t} > \text{et}_j \\ 0, & \text{otherwise} \end{cases}$$

Since smaller tuples are preferred, the solver prioritizes orders that can be delivered on time, then tighter deadlines, shorter delivery distance, higher priority, and smaller order id for deterministic tie-breaking. This reduces the risk of holding packages too long and delaying carried orders.

5) *Pickup Candidate Feasibility*: Before scoring an order, the solver checks whether the shipper can take it. An order is considered only if:

- the order has not been picked up or delivered;
- the shipper still has remaining package slots;
- the current carried weight plus the new order weight does not exceed $W_{\max}$;
- a valid path exists from the shipper to the pickup location;
- a valid path exists from pickup to dropoff.

If any condition is not satisfied, the order is discarded. This avoids infeasible decisions and reduces the number of candidates that need to be scored.

6) *Pickup Scoring*: For each feasible shipper–order pair, the solver computes:

$$d_1 = \text{dist}(\text{pos}_i, \text{pickup}_j)$$

$$d_2 = \text{dist}(\text{pickup}_j, \text{dropoff}_j)$$

$$\text{route_dist}_{ij} = d_1 + d_2$$

$$\text{finish_t} = t + \text{route_dist}_{ij}$$

$$\text{slack}_j = \text{et}_j - \text{finish_t}$$

The expected reward is estimated using the environment reward function:

$$\text{ExpectedReward}_j = \text{delivery_reward}(g_j, \text{finish_t}, T)$$

The value of an order per movement step is:

$$\text{value_rate}_{ij} = \frac{\text{ExpectedReward}_j}{\text{route_dist}_{ij} + 1}$$

The pickup score is represented as:

$$K_{ij}^{\text{pick}} = (l_{ij}^{\text{pick}}, r_{ij}^{\text{reg}} - v_{ij}, s_j, q_{ij}, id_j). \quad (10)$$

Smaller tuples are preferred. Therefore, the solver prioritizes orders that are not late, have high reward per movement step, receive low regional penalty, have reasonable slack, require shorter travel distance, and have smaller ids for stable tie-breaking.

7) *Global Greedy Assignment*: Instead of allowing each shipper to independently choose the nearest order, the solver builds a list of all feasible shipper–order pairs:

$$\text{Candidates} = \{(i, j) \mid \text{shipper}_i \text{ can take order}_j\}$$

Each pair is scored using `pickup_score`. The candidate list is then sorted in ascending order, and the solver greedily assigns orders using the following rules:

- each shipper receives at most one new pickup target;
- each order is assigned to at most one shipper;
- pairs with better scores are selected first.

This is a lightweight online approximation of VRP assignment. It does not solve an exact integer optimization model as OR-Tools would, but it reduces duplicate pursuit of the same order and usually produces a more balanced task distribution than local greedy selection.

8) *Region-Aware Dispatch*: On large maps or maps with many obstacles, a shipper may waste too much time if it travels across the entire map for a low-value order. Therefore, the solver uses region-aware dispatch as a soft penalty mechanism.

The map is divided into four regions:

TopLeft, TopRight, BottomLeft, BottomRight

If the pickup or dropoff location lies outside the shipper's current region, the order receives a `region_penalty`. In the implementation, a pickup in a different region is penalized more heavily than a dropoff in a different region, because traveling far to reach pickup often causes the shipper to miss closer opportunities.

The region penalty is generally computed as:

$$\text{region_penalty} = \text{weight} \cdot \text{raw_penalty}$$

Here, *weight* depends on map size, number of shippers, obstacle ratio, and the average number of free cells each shipper must cover. This is a soft penalty rather than a hard constraint. If an order has sufficiently high reward or a favorable deadline, the solver can still select it.

9) *Large-Map Pruning*: For large maps, the solver applies pruning to remove weak and distant orders. A low-priority order may be discarded if the pickup distance is large, deadline slack is small, or on-time delivery is unlikely. This prevents a shipper from traveling too far for an order whose value does not justify the movement cost, especially when closer opportunities may exist.

Pruning is activated only when the map is large enough or when each shipper must cover a wide free area. This preserves flexibility on small and medium configurations.

10) *Pickup Insertion While Carrying*: If a shipper already has a delivery target but still has remaining capacity, the solver may allow it to pick up an additional order before delivering the carried package. However, this insertion is accepted only if the detour does not significantly harm the deadline of the carried order.

Assume the shipper is carrying order d and considers picking up order p . The direct delivery completion time is:

$$\text{direct_finish} = t + \text{dist}(\text{pos}_i, \text{dropoff}_d)$$

If the shipper picks up the new order first, the carried order completion time becomes:

$$f_{ipd}^{\text{detour}} = t + d(x_i, s_p) + d(s_p, e_d), \quad (11)$$

where f_{ipd}^{detour} is the estimated delivery time of the carried order d after inserting pickup order p into the route. x_i , s_p , and e_d denote the shipper position, pickup location, and drop-off location, respectively, while $d(\cdot, \cdot)$ is the grid shortest-path distance.

The solver rejects pickup insertion if an order that could be delivered on time would become late after the detour:

$$\text{direct_finish} \leq \text{et}_d < \text{detour_finish}$$

In addition, the extra detour must not exceed an adaptive limit:

$$\text{adaptive_limit} = \max \left(2, \min \left(\frac{N}{3}, \frac{\text{dist}(\text{pos}_i, \text{dropoff}_d)}{2} \right) \right). \quad (12)$$

A new pickup is accepted if it is not late according to the score, has priority at least as high as the carried order, or is close enough relative to the current delivery path. This mechanism allows batching when beneficial while protecting the deadline of carried packages.

11) *One-Step Collision Avoidance*: During action generation, the solver maintains a set called `reserved_next`, which stores cells already chosen by previously processed shippers as their next positions in the same timestep. If the best BFS move for a shipper leads to an already reserved cell, the solver tries alternative moves:

$$S, U, D, L, R$$

and selects the valid move that minimizes the remaining distance to the goal.

This is not full multi-agent path planning like CBS, but it is computationally cheap and helps reduce direct collisions in narrow areas or bottlenecks.

12) *Action Generation*: After delivery targets and pickup targets are selected, the solver generates actions for all shippers in priority order. Shippers with urgent deliveries are processed first to reduce the risk of missing deadlines.

If a shipper has a pickup target, it moves toward the pickup location and executes `cargo_op = 1` only when its next position equals the pickup location. This prevents the shipper from accidentally picking up another order at the current cell. If a shipper has a delivery target, it moves toward the dropoff location and executes `cargo_op = 2` when it reaches the delivery cell. If no target is available, the shipper stays still with action $(S, 0)$.

13) *Complexity Analysis*: Let C be the number of shippers, A the number of active orders at the current timestep, F the number of free cells on the map, and Q the number of distinct path queries cached so far. A single BFS call has complexity:

$$O(F)$$

In the worst case, the solver scores all shipper-order pairs, so the number of scoring operations is:

$$O(C \cdot A)$$

Ignoring cache benefits, the worst-case path computation cost per timestep can be approximated as:

$$O(C \cdot A \cdot F)$$

With caching, many distance queries are reused, so the practical runtime is much lower. The additional memory usage depends on the number of cached position pairs:

$$O(Q)$$

14) *Strengths and Limitations*: The strengths of the VRP-style solver are determinism, interpretability, no external library dependency, and good runtime performance. Global assignment reduces the chance that multiple shippers pursue the same order, while reward-, deadline-, and capacity-aware scoring helps the algorithm choose better tasks than nearest-order greedy. Region penalty, pickup insertion, and reserved next-cell avoidance further improve behavior on large maps or maps with bottlenecks.

However, this method is still a heuristic, not an exact VRP solver. It does not optimize full multi-stop routes as a complete OR-Tools model would, does not perform long-horizon lookahead for future orders, and only handles collisions at the one-step level. Therefore, in scenarios with strong multi-agent congestion or where longer coordinated planning is required, it may be less effective than MAPD-CBS.

C. ACO

ACO is implemented as an online solver for the Multi-Agent Package Delivery problem. The method name in the code is *ACO_Strict_Online*. The algorithm is inspired by Ant Colony Optimization, where order-selection decisions are evaluated using a combination of current heuristic information and pheromone accumulated from previous decisions.

Unlike classical ACO, which usually constructs solutions through multiple iterations of a colony, this version is adapted to an online environment. At each timestep, the solver only observes the orders that have already appeared, evaluates feasible pickup or delivery choices, and then uses BFS to generate the next movement step. Since future orders are unknown, the solver does not construct a fixed route for the entire simulation. Instead, it replans after every timestep.

1) *Design Objective*: The objective of the ACO solver is to maximize *net reward* by balancing expected reward, on-time delivery feasibility, and movement cost. For each shipper i and order j , the solver considers:

- distance from the shipper to the pickup location;
- distance from pickup to dropoff;
- deadline and deadline slack;
- expected reward at the estimated delivery time;
- order priority;
- remaining package slots and carrying capacity;
- hotspot bonus around the pickup location;
- pheromone value associated with the order.

This design allows the solver to select orders not only based on proximity, but also based on reward, deadline feasibility, and the current state of the shipper.

2) *Overall Pipeline*: At each timestep, the ACO solver performs the following steps:

- 1) Receive the current observation from the environment.
- 2) Extract active orders and apply pheromone evaporation.
- 3) Update dynamic hotspot beliefs from unpicked orders.
- 4) Select delivery targets for shippers carrying packages.
- 5) Generate feasible pickup candidates for idle shippers.
- 6) Score each shipper-order pair using the ACO score.
- 7) Apply global greedy assignment to select pickup targets.

- 8) Generate movement actions using BFS next moves.
- 9) Reserve next cells to reduce direct collisions.
- 10) Execute pickup or delivery if the shipper reaches the corresponding cell.
- 11) Submit actions to the environment and repeat until the simulation ends.

The overall pipeline can be represented as:

```

Observation → PheromoneUpdate
              → HotspotUpdate
              → DeliverySelection
              → CandidateGeneration
              → ACOAssignment
              → BFSPlanning
              → CollisionAvoidance
              → Execute

```

3) *BFS Routing and Caching*: The solver uses BFS to compute shortest paths on the obstacle grid. Since each movement step has the same cost, BFS is suitable for computing the true shortest distance between two cells.

For two positions s and g , the solver stores:

$$PathCache[(s, g)] = (dist(s, g), first_move(s, g))$$

Here, $dist(s, g)$ is the length of the shortest path, and $first_move(s, g)$ is the first movement action along that path. If no valid path exists, the distance is set to INF , and the corresponding candidate is removed from consideration.

Caching reduces repeated BFS computations, since the same pair of positions may be queried multiple times during order scoring and action generation. The cache size is also adjusted according to map size to avoid excessive memory usage.

4) *Pheromone Update*: Pheromone is stored by task type and order id:

$$\tau(task, j)$$

In the implementation, the pheromone key is:

$$(task_type, order_id)$$

The default pheromone value is 1.0. At each timestep, pheromone evaporates with rate ρ :

$$\tau \leftarrow \tau(1 - \rho)$$

The current implementation uses:

$$\alpha = 1.0, \quad \beta = 2.2, \quad \rho = 0.05$$

Pheromone is bounded by:

$$\tau_{\min} = 0.2, \quad \tau_{\max} = 15.0$$

If an order is no longer active, its pheromone entry is removed. When the solver executes a promising pickup or delivery action, the corresponding pheromone is reinforced:

$$\tau \leftarrow \min(\tau_{\max}, \tau + amount)$$

This mechanism allows the solver to remember useful decisions while still adapting to the changing online environment.

5) *Pickup Candidate Feasibility*: Before scoring an order, the solver checks whether the shipper can carry it. An order is considered only if:

- the order has not been picked up or delivered;
- the shipper still has remaining package slots;
- the current carried weight plus the new order weight does not exceed $W_{\max}$;
- a valid path exists from the shipper to the pickup location;
- a valid path exists from pickup to dropoff.

For large maps, the solver also uses Manhattan distance to quickly filter out orders that are too far before running BFS. This reduces computation cost and runtime.

6) *Pickup Heuristic*: For each feasible shipper–order pair, the solver computes:

$$d_1 = \text{dist}(\text{pos}_i, \text{pickup}_j)$$

$$d_2 = \text{dist}(\text{pickup}_j, \text{dropoff}_j)$$

$$\text{finish}_t = t + d_1 + d_2$$

$$\text{slack}_j = \text{et}_j - \text{finish}_t$$

The expected reward is computed using the environment reward function:

$$\text{ExpectedReward}_j = \text{delivery_reward}(j, \text{finish}_t, T)$$

The solver also estimates travel cost based on the current carried weight:

$$\text{TravelCost} = \text{cost}(d_1) + \text{cost}(d_2)$$

The estimated net profit is then:

$$\begin{aligned} \text{net_profit} &= \text{ExpectedReward}_j \\ &\quad - \text{TravelCost} \\ &\quad + \text{HotspotBonus} \\ &\quad + \text{UrgencyBonus} \end{aligned}$$

If the order can still be delivered on time, net_profit is increased. If the order is estimated to be late, it is penalized. In the implementation, orders with non-negative slack receive a multiplier, while late orders are discounted:

$$\text{net_profit} \leftarrow \begin{cases} 1.55 \cdot \text{net_profit}, & 0 \leq \text{slack} \leq 30 \\ 1.25 \cdot \text{net_profit}, & \text{slack} > 30 \\ 0.65 \cdot \text{net_profit}, & \text{slack} < 0 \end{cases}$$

The final heuristic is normalized by travel distance:

$$\eta_{ij} = \frac{\text{net_profit}}{1 + d_1 + 0.5d_2}$$

For large maps, the solver penalizes pickup distance more heavily:

$$\eta_{ij} = \frac{\text{net_profit}}{1 + 1.5d_1 + 0.4d_2}$$

7) *ACO Pickup Score*: The pickup score for shipper i and order j is computed as:

$$\text{score}_{ij} = \tau_j^\alpha \cdot \eta_{ij}^\beta$$

where:

- τ_j is the pheromone value of order j ;
- η_{ij} is the heuristic value of the shipper–order pair;
- α controls the influence of pheromone;
- β controls the influence of heuristic information.

With $\alpha = 1.0$ and $\beta = 2.2$, the solver emphasizes the current heuristic more strongly than past pheromone. This is appropriate for an online environment where the system state changes continuously.

8) *Delivery Target Selection*: For a shipper carrying packages, the solver prioritizes which order to deliver first. For each order j in shipper i 's bag, it computes:

$$\text{arrive}_t = t + \text{dist}(\text{pos}_i, \text{dropoff}_j)$$

$$\text{slack}_j = \text{et}_j - \text{arrive}_t$$

If $\text{slack}_j \geq 0$, the order can still be delivered on time and is favored. If $\text{slack}_j < 0$, the order is strongly penalized so that the shipper does not keep chasing low-value late deliveries before urgent feasible ones.

The delivery score considers:

- deadline urgency;
- reward per movement step;
- order priority;
- distance to dropoff;
- order id for deterministic tie-breaking.

9) *Dynamic Hotspot*: The solver builds dynamic hotspots from orders that have not yet been picked up. Each waiting order contributes to the hotspot value at its pickup location:

$$\text{Hotspot}[\text{pickup}_j] \leftarrow \text{Hotspot}[\text{pickup}_j] + 1$$

When scoring an order, if its pickup location is near an area with many other waiting orders, the solver adds a hotspot bonus:

$$\text{HotspotBonus} = \sum_h \frac{\text{weight}_h}{1 + \text{dist}(\text{pos}, h)}$$

Only hotspots within a small Manhattan radius are considered, and the bonus is capped to avoid dominating the score. This encourages shippers to move toward areas with more potential orders, improving future pickup opportunities.

10) *Global Greedy Assignment*: After computing ACO scores, the solver assigns pickup targets using global greedy assignment. At each iteration, the solver searches for the best shipper–order pair among available shippers and unassigned orders.

The assignment rules are:

- each shipper receives at most one pickup target;
- each order is pursued by at most one shipper;
- the pair with the best ACO score is selected first;
- priority, distance, deadline, and order id are used for tie-breaking.

This reduces the chance that multiple shippers move toward the same order and produces a more balanced task distribution than local greedy assignment.

11) *Large-Map Pruning*: For large maps, the solver limits the number of pickup candidates considered for each shipper using the `_candidate_limit` function. As the map size grows, the maximum number of candidates is reduced to control scoring and assignment cost.

The candidate limits are:

Map size	Candidate limit
$N \geq 100$	10
$N \geq 80$	12
$N \geq 50$	18
$N \geq 30$	32
$N < 30$	80

The solver also removes orders whose pickup locations are too far away or whose lateness risk is too high, especially when priority is low. This prevents shippers from crossing the map for low-value orders while missing better nearby opportunities.

12) *Extra Pickup Before Delivery*: On small and medium maps, the solver may allow a shipper carrying packages to pick up an additional order before delivering. This mechanism is enabled only when:

$$12 \leq N \leq 30$$

An extra pickup is considered only if:

- the shipper still has remaining package slots;
- the new order has not been picked up;
- the pickup location is close enough to the shipper;
- the additional pickup does not significantly delay the current delivery.

In the current implementation, if `pickup_dist` is the distance from the shipper to the new order and `delivery_dist` is the distance to the current delivery target, an extra pickup is considered only when:

$$pickup_dist \leq 3$$

and

$$pickup_dist + 2 < delivery_dist$$

This helps use remaining capacity and increases batching opportunities while limiting the risk of delaying carried orders.

13) *One-Step Collision Avoidance*: During action generation, the solver maintains a set of cells already selected by other shippers as their next positions:

$$OccupiedNextPositions$$

If the best BFS move leads to an occupied next position, the solver searches for an alternative direction among:

$$U, D, L, R$$

If no valid alternative exists, the shipper stays in place with move `S`. This is not a full multi-agent path planning algorithm like CBS, but it is lightweight and helps reduce direct collisions within a timestep.

14) *Action Generation*: After delivery targets and pickup assignments are selected, the solver generates an action for each shipper.

If a shipper has a delivery target, it moves toward the dropoff location using the BFS next move. If its position after movement matches the destination of an order in its bag, the solver executes:

$$cargo_op = 2$$

If a shipper has a pickup target, it moves toward the pickup location. If its position after movement contains a feasible order, the solver executes:

$$cargo_op = 1$$

If the shipper has no delivery or pickup target, it stays still:

$$action = (S, 0)$$

where:

- $cargo_op = 0$: no cargo operation;
- $cargo_op = 1$: pick up an order;
- $cargo_op = 2$: deliver an order.

This ensures that pickup and delivery operations are only performed when the shipper actually reaches the corresponding cell.

15) *Complexity Analysis*: Let C be the number of shippers, A the number of active orders at the current timestep, F the number of free cells on the map, Q the number of distinct path queries cached so far, and K the maximum number of candidates retained by `_candidate_limit`.

A single BFS call has complexity:

$$O(F)$$

After candidate limiting, the number of shipper–order pairs evaluated is approximately:

$$O(C \cdot K)$$

Ignoring cache benefits, the worst-case cost per timestep is:

$$O(C \cdot K \cdot F)$$

With path caching, candidate pruning, and fast Manhattan filtering, the practical runtime is usually much lower than the worst-case bound. The auxiliary memory usage mainly comes from path cache, pheromone, and hotspot data:

$$O(Q + A)$$

16) *Strengths and Limitations*: The ACO solver is fast, independent of external libraries, suitable for online execution, and easy to interpret. Pheromone helps the solver remember useful decisions, heuristic scoring evaluates orders based on reward, deadline, and distance, and BFS caching reduces pathfinding cost.

However, the method is still a heuristic and does not guarantee global optimality. The current pheromone model is associated with task and order ids, rather than edges or full paths as in classical ACO. In addition, collision avoidance is only handled at the one-step level, so in complex bottlenecks it may be less effective than specialized multi-agent path planning methods such as CBS.

D. MAPD-CBS

MAPD-CBS is a hybrid method for the online Multi-Agent Package Delivery problem. It is not a pure CBS solver. Instead, it combines reward- and deadline-aware task assignment, online hotspot/surge inference, min-cost flow matching for difficult cases, CBS with low-level A* for small and medium instances, and reserved-next-cell planning for large instances.

Since future orders are unknown, the solver follows a rolling-horizon strategy. At each timestep, it plans using only the current observation, executes exactly one step, observes the environment again, and replans.

1) *Overall Pipeline*: The main pipeline of MAPD-CBS at each timestep is:

Observation $\rightarrow$ Hotspot/Surge Inference
 $\rightarrow$ Task Assignment $\rightarrow$ Path Planning
 $\rightarrow$ Conflict Handling $\rightarrow$ Execute One Step.

More specifically, the solver performs the following steps:

- 1) Refresh public information from the observation: N, C, G, T , grid, orders, shippers, and current time.
- 2) Update the `HotspotTracker` using newly revealed orders in `new_order_ids`.
- 3) Estimate surge intensity from the recent order arrival rate.
- 4) Assign delivery targets to shippers carrying packages.
- 5) Assign pickup targets to shippers with remaining capacity.
- 6) For small/medium instances, plan paths using CBS with low-level A*.
- 7) For large instances, use cached shortest-path next moves and reserved cells.
- 8) Generate actions, submit them to the environment, and repeat at the next timestep.

2) *Online State Refresh and Mode Selection*: The solver only uses public observation data. If the grid object changes, all path caches are cleared to avoid using stale path information. The main parameters are updated from the observation:

$$N, C, G, T$$

Large mode is enabled when the number of shippers or orders is sufficiently large:

$$large_mode = (C \geq 12) \vee (G \geq 500)$$

In small/medium mode, the solver uses CBS to handle path conflicts. In large mode, full CBS can be computationally expensive, so the solver switches to a faster strategy: candidate filtering with Manhattan distance, cached BFS next moves, and reserved-next-position collision avoidance.

3) *Shortest Path Cache*: MAPD-CBS uses BFS to compute shortest paths on the obstacle grid. The solver stores three types of cache:

$$PathCache[(s, g)] = path(s, g)$$

$$DistanceCache[(s, g)] = dist(s, g)$$

$$NextMoveCache[(s, g)] = first_move(s, g)$$

Here, $path(s, g)$ is the shortest path from s to g , $dist(s, g) = |path| - 1$, and $first_move(s, g)$ is the first movement action along that path. Caching significantly reduces repeated BFS calls when many shippers and orders generate similar path queries across timesteps.

4) *Delivery Assignment*: For a shipper carrying packages, the solver selects a delivery target using delivery ranking. For shipper i and order j :

$$finish_t = t + dist(pos_i, dropoff_j)$$

$$slack_j = et_j - finish_t$$

$$rps_j = \frac{delivery_reward(g_j, finish_t, T)}{\max(dist(pos_i, dropoff_j), 1)}$$

In small/medium mode, the delivery rank favors reachable orders, high reward per step, on-time delivery, tight deadlines, short distance, and high priority:

$$delivery_rank = (unreachable, -rps_j, late_flag, \max(slack_j, 0), distance, -p_j, id_j). \quad (13)$$

Large mode uses a similar ranking but emphasizes reward per movement step and delivery feasibility. This reduces the risk of holding packages too long and missing deadlines.

5) *Pickup Scoring*: For an unpicked order, the solver evaluates pickup feasibility and value based on distance, reward, deadline, priority, hotspot signal, and the effect on currently carried orders. For shipper i and order j :

$$route_dist_{ij} = dist(pos_i, pickup_j) + dist(pickup_j, dropoff_j)$$

$$finish_t = t + route_dist_{ij}$$

$$ExpectedReward_j = delivery_reward(g_j, finish_t, T)$$

$$rps_{ij} = \frac{ExpectedReward_j}{\max(route_dist_{ij}, 1)}$$

In small/medium mode, the pickup rank includes:

$$pickup_rank = (unreachable, insertion_penalty, region_penalty, -rps, -0.03 \cdot reward, late_flag, pickup_distance, -p_j, slack_j, delivery_distance, id_j). \quad (14)$$

If the shipper is already carrying an order, `insertion_penalty` measures the additional detour caused by picking up the new order before delivering the best carried order:

$$\pi_{ij}^{ins} = \max(0, d(x_i, s_j) + d(s_j, e_d) - d(x_i, e_d)). \quad (15)$$

This allows batching when useful while discouraging pickups that significantly delay current deliveries.

6) *Hotspot and Surge Inference*: The solver does not directly read hidden hotspot or surge windows from the configuration. Hotspots are inferred from newly revealed pickup locations, while surge intensity is estimated from the number of new orders in a recent sliding window.

The surge score is computed as:

$$recent_rate = \frac{new_orders_in_last_60_steps}{60}$$

$$base_rate = \frac{G}{T}$$

$$surge_score = clamp\left(\frac{recent_rate/base_rate - 1}{3}, 0, 1\right)$$

The raw hotspot bonus is:

$$raw_hotspot = HotspotScore(pickup_j) \cdot (1 + 0.20 \cdot surge_score)$$

For large or difficult instances, the solver balances hotspot value using local demand and supply:

$$confidence = \min(1, demand/5)$$

$$balance = \frac{demand}{demand + supply + 1}$$

$$balanced_hotspot = raw_hotspot \cdot confidence \cdot balance$$

Here, *demand* is the number of active unpicked orders near the hotspot, while *supply* is the number of nearby shippers. This encourages underserved hotspots instead of sending more shippers to already-covered areas.

7) *Region Penalty*: The solver applies a light region penalty in selected medium configurations, especially when $N = 18$. The map is divided into four quadrants. If an order pickup or dropoff is outside the shipper's current region, the order receives a penalty:

$$r_{ij}^{reg} = w_{reg} \left(1.0 \mathbb{I}[\rho(s_j) \neq \rho(x_i)] + 0.35 \mathbb{I}[\rho(e_j) \neq \rho(x_i)] \right). \quad (16)$$

where r_{ij}^{reg} is the region penalty for assigning order j to shipper i , w_{reg} is the adaptive region weight, $\rho(\cdot)$ maps a cell to its region, x_i is the current position of shipper i , and s_j and e_j denote the pickup and drop-off locations of order j , respectively.

The weight depends on obstacle ratio and the average number of free cells each shipper must cover. This is a soft penalty, not a hard constraint, so cross-region orders can still be selected if their reward or deadline is favorable.

8) *Adaptive Min-Cost Flow Matching*: When workload is high or the instance is difficult, the solver uses min-cost flow to assign pickup targets. The matching graph is:

$$source \rightarrow shipper \rightarrow order \rightarrow sink$$

Each shipper and each order has capacity 1. The edge value between a shipper and an order is:

$$\begin{aligned} value = & 10.0 \cdot rps \cdot hotspot_factor + 0.03 \cdot reward \\ & + 0.50 \cdot priority - 0.08 \cdot pickup_distance \\ & - 0.04 \cdot delivery_distance - 0.20 \cdot lateness \\ & - 1.00 \cdot insertion_penalty - 0.60 \cdot region_penalty. \end{aligned} \quad (17)$$

Min-cost flow uses negative edge costs:

$$cost(i, j) = -1000 \cdot value(i, j)$$

The algorithm augments only when a negative-cost path exists, meaning the assignment increases total value. Flow is not enabled all the time. It is activated based on difficulty, active pressure, obstacle ratio, and free cells per shipper:

$$avg_route_len = \frac{4N}{3}$$

$$budget_per_order = \frac{T \cdot C}{G}$$

$$difficulty = \frac{avg_route_len}{budget_per_order}$$

$$active_pressure = \frac{active_orders}{C}$$

Flow is used for very hard cases, bottleneck-like instances, sparse large maps, or severe order queues.

9) *Beam Lookahead*: For some small configurations, especially $N = 12$, the solver uses a lightweight beam lookahead to evaluate not only the first order but also the follow-up opportunities after completing it:

$$lookahead = first_value + 0.35 \cdot \sum top_2(followup_values)$$

This helps avoid selecting an order that looks good immediately but leaves the shipper in a poor position for subsequent orders.

10) *Sticky Pickup*: In large mode, the solver uses sticky pickup to reduce frequent target switching. A previous pickup target can be kept for up to 8 timesteps if the new target is not clearly better:

$$current_value \leq 1.25 \cdot \max(old_value, sticky_value)$$

Sticky pickup is removed if the order has been picked up or delivered, becomes too old, the shipper can no longer carry it, or the shipper must prioritize delivery. This reduces zig-zag behavior and unnecessary travel.

11) *CBS with Low-Level A**: In small/medium mode, the solver uses Conflict-Based Search to plan paths for multiple shippers. CBS has two levels:

- Low-level: constrained A* over $(position, time)$ states.
- High-level: conflict detection and constraint generation.

Low-level A* uses:

$$g = \text{path length so far}$$

$$h = dist(current, target)$$

$$f = g + h$$

CBS handles two types of conflicts:

$$\text{VertexConflict} : pos_i(t) = pos_j(t)$$

$$\text{EdgeConflict} : pos_i(t) = pos_j(t+1) \wedge pos_i(t+1) = pos_j(t)$$

When a conflict is found, CBS branches by adding a vertex or edge constraint to one of the conflicting shippers, then replans only that shipper. The number of expanded CBS nodes is limited by `max_cbs_nodes` to avoid timeout.

12) *Large-Mode Fast Planning*: Full CBS does not scale well in large mode. The solver therefore uses a faster strategy:

- shortlist orders using Manhattan distance;
- run exact BFS only for a small number of strong candidates;
- use cached next moves toward the selected target;
- reserve next cells to reduce one-step collisions.

Shippers are processed by task urgency: deliveries before pickups, then deadline, priority, and distance. If the best next cell is already reserved, the solver tries alternative moves and selects the one closest to the target.

13) *Fallback Strategy*: If CBS cannot find a conflict-free plan within the node budget, the solver falls back to a greedy one-step planner. The fallback chooses a valid move closest to the target while avoiding already reserved cells. This ensures that the solver always produces valid actions even when CBS fails or exceeds its search budget.

14) *Complexity Analysis*: Let C be the number of shippers, A the number of active orders, F the number of free cells on the map, and K the number of shortlisted candidates.

A BFS/A* search costs approximately:

$$O(F)$$

Greedy assignment in small/medium mode may require:

$$O(C \cdot A \cdot F)$$

in the worst case, although caching substantially reduces the practical cost.

Large mode uses prefiltering:

$$O(C \cdot A) + O(C \cdot K \cdot F)$$

CBS has worst-case exponential complexity in the number of conflicts, since each conflict can branch into additional constraints. Therefore, CBS is only used in small/medium mode and the number of expanded nodes is capped.

15) *Strengths and Limitations*: The main strength of MAPD-CBS is its combination of task assignment and path planning. The solver jointly considers reward, deadline, priority, hotspot signal, region penalty, insertion penalty, and path conflicts. Rolling-horizon replanning allows it to adapt to online orders and changing workload over time. Min-cost flow improves assignment quality in difficult cases, while CBS reduces conflicts on small and medium maps.

The main limitation is higher complexity and runtime compared with lightweight heuristics such as Greedy BFS or VRP-style assignment. CBS does not scale well when the number

of shippers is large, so large mode must rely on approximate planning. Moreover, the solver still cannot know future orders, so decisions made at the current timestep may not be globally optimal over the whole simulation.

V. IMPLEMENTATION DETAILS

A. Solver Architecture

The solvers are implemented as online algorithms that interact directly with the environment through an observe-decide-execute loop. At each time step, a solver receives the current state from the environment, builds target assignments for shippers, generates valid actions, and then calls `env.step(actions)` to move to the next simulation step.

The general solver pipeline is:

$$\begin{aligned} \text{Observe} &\rightarrow \text{UpdateState} \\ &\rightarrow \text{AssignTasks} \\ &\rightarrow \text{PlanMoves} \\ &\rightarrow \text{Execute} \end{aligned}$$

Here, `Observe` reads the current environment state, `UpdateState` updates auxiliary information such as path caches or hotspot trackers, `AssignTasks` selects pickup or delivery targets for shippers, `PlanMoves` generates the next movement step, and `Execute` submits the actions to the environment.

B. Online Observation Handling

At each timestep, the solver only uses public information provided by the environment. This information includes the current time t , the grid map, shipper states, revealed orders, newly generated orders in `new_order_ids`, and the simulation termination flag.

Since the problem is online, the solver does not access future orders and does not directly use hidden surge or hotspot information. Instead, it infers the system state from already revealed orders. This makes the solver consistent with the evaluation setting and avoids relying on information that is not available during actual execution.

C. Path Planning

Path planning is mainly performed using BFS on the obstacle grid. Since each movement step has the same cost, BFS can compute the shortest path between two valid cells. In practice, the solver usually only needs the first move along the shortest path, so BFS results are stored in two caches:

$$\text{DistanceCache}[(s, g)] = \text{dist}(s, g)$$

$$\text{NextMoveCache}[(s, g)] = \text{first_move}(s, g)$$

For more advanced methods such as MAPD-CBS, path planning may combine A* and Conflict-Based Search to detect and resolve conflicts among multiple shippers. However, for large configurations, the solver favors lightweight strategies such as reserved next-position to reduce computational overhead.

D. Order Scoring

Each order is evaluated based on distance, deadline, expected reward, priority, weight, and shipper feasibility. For a shipper i and an order j , the estimated route distance is computed as:

$$route_dist_{ij} = dist(pos_i, pickup_j) + dist(pickup_j, dropoff_j)$$

$$finish_t = t + route_dist_{ij}$$

$$slack_j = et_j - finish_t$$

An order is preferred if it has high reward, sufficient $slack_j$, short travel distance, and fits the shipper's remaining carrying capacity. The value of an order per movement step can be computed as:

$$value_rate_{ij} = \frac{ExpectedReward_j}{route_dist_{ij} + 1}$$

For orders already carried by a shipper, the algorithm prioritizes deliveries with urgent deadlines or high risk of lateness. This reduces the chance of holding packages for too long and keeps shipper capacity available for future orders.

E. Surge/Hotspot Strategy

Although surge and hotspot information is not directly provided to the solver, the algorithm can infer high-demand areas from newly revealed orders. Whenever a new order appears, its pickup location is added to a hotspot tracker. Cells or regions with many recent order arrivals are treated as high-demand areas.

The inferred hotspot information is used in two ways. First, when a shipper is idle, the solver may reposition it toward a hotspot area to increase the chance of serving future orders. Second, during order scoring, orders near active hotspots or in high-load regions may receive higher priority if they provide good reward and do not cause excessive detours.

This strategy helps the solver adapt to surge periods without accessing hidden environment configuration. However, hotspot preference is only used as a soft heuristic, since the solver must still balance repositioning, on-time delivery, and serving currently available orders.

VI. EXPERIMENTAL RESULTS

A. Net Reward Comparison

Table II reports the *net reward* obtained by the four methods on the six Phase 1 configurations. The *net reward* is the primary evaluation metric, representing the total delivery reward after subtracting movement cost.

The results show that MAPD-CBS achieves the highest total *net reward*, with 5713.45 points. It performs particularly well on larger configurations such as C5 and C6, where the number of shippers, orders, and congestion level increase. VRP achieves the second-best total score due to its stable task assignment strategy, while Greedy BFS performs reasonably well on small and medium configurations. ACO obtains strong results on C4 and C5 but drops significantly on C6, indicating lower stability at larger scales.

Config	Greedy BFS	VRP	ACO	MAPD-CBS
C1	247.67	267.12	266.61	267.18
C2	418.79	364.45	440.57	472.20
C3	840.29	856.32	694.04	903.26
C4	934.25	776.64	1023.68	1039.93
C5	804.27	1224.08	1306.05	1405.97
C6	1222.17	1328.42	1461.98	1624.91
Total	4467.43	4817.02	5192.94	5713.45

TABLE II: Net reward comparison on Phase 1 configurations

B. Delivery Rate and On-Time Rate

Table III summarizes the number of delivered orders, delivery rate, and on-time rate of each method across all six configurations.

Method	Delivered	Delivery Rate	On-Time Rate
Greedy BFS	241/320	75.31%	84.23%
VRP-OrTools	282/320	88.13%	79.79%
ACO	284/320	88.75%	84.51%
MAPD-CBS	296/320	92.50%	88.85%

TABLE III: Delivery rate and on-time rate on Phase 1 configurations

MAPD-CBS achieves the highest delivery rate at 92.50% rate at 88.85%. This indicates that the method not only delivers more orders but also handles deadlines more effectively. ACO achieves the second-highest delivery rate at 88.75%, demonstrating strong scalability and effective online decision making across different configurations. Its on-time rate of 84.51% is competitive, although it is slightly lower than MAPD-CBS due to increased delivery pressure in larger instances. VRP also delivers many orders, but its on-time rate is lower because more deliveries are completed after their deadlines. Greedy BFS has the lowest delivery rate, particularly when the number of orders and traffic congestion increase.

C. Runtime Analysis

Table IV presents the total runtime of each method across all six Phase 1 configurations.

Method	Runtime (s)
Greedy BFS	7.70
VRP-OrTools	4.80
ACO	3.90
MAPD-CBS	12.60

TABLE IV: Runtime over all Phase 1 configurations

ACO and VRP have the lowest runtime because they rely on lightweight heuristic decisions. Greedy BFS is slightly slower due to frequent BFS queries, although path caching reduces repeated computation. MAPD-CBS has the highest runtime because it combines task assignment, conflict handling, and

path planning. However, its runtime of 14.26 seconds remains acceptable and is justified by its superior solution quality.

VII. DISCUSSION

The experimental results show that each method has a different behavior when evaluated by net reward, delivery rate, on-time rate, and runtime. Therefore, the performance of the methods should not be judged by a single metric only. A method with high delivery rate may not always deliver orders on time, while a method with high solution quality may require higher computational cost. This reflects the main trade-off in dynamic delivery systems: the system must balance planning quality and real-time decision-making.

Greedy BFS is the simplest method among the four approaches. Its main advantage is that it is easy to implement and can make decisions based on local heuristic information. However, the results show that this simplicity limits its overall performance. Greedy BFS obtains the lowest total net reward, with 4467.43 points, and also has the lowest delivery rate, delivering only 241 out of 320 orders. This result suggests that local decision-making is not sufficient when the environment becomes more complex. Since Greedy BFS mainly selects the currently best-looking action, it may ignore better future opportunities, global shipper distribution, or the long-term effect of assigning an order to a particular shipper.

However, Greedy BFS is not completely ineffective. Its on-time rate is 84.23%, which is higher than the on-time rate of VRP. This means that although Greedy BFS delivers fewer orders, a relatively large portion of its completed deliveries are still finished before the deadline. In other words, Greedy BFS tends to be more conservative: it may fail to complete many orders, but the orders it does complete are not always late. This makes Greedy BFS suitable as a simple baseline or fallback method, especially in small scenarios where the number of shippers and orders is limited.

The VRP-based method improves the delivery rate compared with Greedy BFS. It delivers 282 out of 320 orders, achieving a delivery rate of 88.13%, which is much higher than the 75.31% delivery rate of Greedy BFS. Its total net reward is also higher, reaching 4817.02 points. This shows that modeling the problem as a vehicle routing problem helps the system consider multiple shippers and orders together instead of making purely local decisions. As a result, VRP can produce a better task assignment strategy than Greedy BFS.

Nevertheless, VRP has the lowest on-time rate among the four methods, with only 79.79%. This is an important trade-off. Although VRP can complete more deliveries, it does not handle deadlines as effectively as the other methods. This may happen because the VRP formulation focuses more on assigning and routing orders efficiently, but the deadline constraint may not be strong enough to prioritize urgent deliveries in all cases. Therefore, VRP is useful for improving delivery quantity, but it still needs better deadline-aware mechanisms to improve delivery timeliness.

ACO achieves a strong balance between solution quality and runtime. It obtains the second-highest total net reward,

with 5192.94 points, and delivers 284 out of 320 orders, corresponding to a delivery rate of 88.75%. Its on-time rate is 84.51%, which is slightly higher than Greedy BFS and clearly higher than VRP. These results show that ACO can improve the quality of delivery planning while maintaining competitive deadline performance.

The most notable advantage of ACO is its runtime. ACO has the lowest runtime among the four methods, requiring only 3.90 seconds across all Phase 1 configurations. This makes it the most efficient method in terms of computational cost. Compared with Greedy BFS, ACO achieves both higher net reward and lower runtime. Compared with MAPD-CBS, ACO is much faster, although its total reward and delivery quality are lower. Therefore, ACO is a good choice when the system needs fast online decision-making but still requires better performance than a simple greedy baseline.

However, ACO also has some limitations. Its performance is not completely stable across all configurations. For example, although ACO performs strongly in C4 and C5, it obtains a lower score in C3 compared with the other methods. This indicates that ACO may be sensitive to configuration characteristics and parameter settings, such as pheromone influence, heuristic weight, evaporation rate, number of ants, and number of iterations. Therefore, ACO can be effective, but it may require careful parameter tuning to achieve stable performance in different environments.

MAPD-CBS achieves the best overall performance in the experiments. It obtains the highest total net reward, with 5713.45 points, the highest delivery rate, delivering 296 out of 320 orders, and the highest on-time rate, with 88.85%. These results show that MAPD-CBS not only completes more orders but also handles deadlines better than the other methods. Its advantage is especially clear in larger configurations such as C5 and C6, where the number of orders, shippers, and possible interactions increases.

The strong performance of MAPD-CBS comes from its ability to consider multi-agent path planning and conflict resolution. In a grid-based delivery environment, multiple shippers may move through the same area and block one another if their routes are planned independently. MAPD-CBS addresses this issue by resolving conflicts between agents, which helps reduce inefficient movements and improves the feasibility of the final plan. This explains why MAPD-CBS performs well in both reward-based and delivery-based metrics.

The main limitation of MAPD-CBS is runtime. It requires 12.60 seconds, which is the highest runtime among the evaluated methods. This is expected because MAPD-CBS has to handle task assignment, path planning, and conflict resolution at the same time. When the number of agents or conflicts increases, the search process becomes more expensive. Therefore, MAPD-CBS trades computational efficiency for better coordination and higher solution quality. This trade-off is acceptable when delivery quality is the main priority, but it may become a limitation in systems with very strict real-time constraints.

Overall, the results show that no method is best in every

aspect. Greedy BFS is simple and can be used as a baseline, but it has the lowest total reward and delivery rate. VRP improves delivery quantity, but its lower on-time rate shows that it needs better deadline handling. ACO provides the best runtime and strong overall performance, making it a balanced method for fast decision-making. MAPD-CBS achieves the best net reward, delivery rate, and on-time rate, but it requires the highest computational cost.

These findings suggest that a practical delivery system should consider the operating context before selecting a method. If the system prioritizes fast response time, ACO is a suitable choice. If the system prioritizes delivery quality, deadline satisfaction, and multi-agent coordination, MAPD-CBS is more appropriate. If the system needs a simple baseline, Greedy BFS can be used. If the goal is to improve task assignment with a clearer optimization structure, VRP is useful, but it should be improved with stronger deadline-aware constraints. A promising direction is to combine these methods, for example using ACO or VRP for fast task assignment and applying MAPD-CBS only in congested areas where path conflicts are likely to occur.

VIII. CONCLUSION

This paper studied the problem of shipper assignment and route planning in a dynamic delivery system. The main objective was to assign orders to available shippers and generate delivery routes that maximize net reward while considering delivery deadlines, movement cost, and interactions among multiple agents. Four methods were implemented and evaluated: Greedy BFS, a VRP-based method, Ant Colony Optimization, and MAPD-CBS.

The methods were compared on six Phase 1 configurations using four main metrics: net reward, delivery rate, on-time rate, and runtime. The results show that each method has its own advantages and limitations. Greedy BFS is simple and easy to implement, but its local decision-making strategy leads to lower total reward and delivery rate. VRP improves task assignment and completes more orders than Greedy BFS, but its on-time rate is the lowest among the evaluated methods. ACO achieves a good balance between solution quality and computational efficiency, obtaining the lowest runtime and the second-highest total net reward. MAPD-CBS achieves the best overall delivery performance, with the highest net reward, delivery rate, and on-time rate.

From these results, the main conclusion is that dynamic delivery optimization requires a balance between solution quality and computational cost. MAPD-CBS is the most effective method when the priority is high-quality planning, multi-agent coordination, and deadline satisfaction. ACO is the most suitable method when runtime is the main concern but the system still needs competitive solution quality. VRP provides a useful global assignment framework, while Greedy BFS remains valuable as a simple baseline or fallback strategy.

In future work, the system can be improved by combining the strengths of different methods. For example, ACO or VRP can be used to generate fast initial assignments, while

MAPD-CBS can be applied selectively to resolve conflicts in congested areas. In addition, the VRP formulation can be extended with stronger deadline-aware constraints, and ACO can be improved through adaptive parameter tuning. More experiments on larger maps, higher order arrival rates, and more complex traffic conditions should also be conducted to evaluate the scalability and robustness of the proposed approaches.

ACKNOWLEDGMENT

The authors would like to thank the course instructors and teaching assistants for providing the problem statement, benchmark configurations, and evaluation framework for this project.

CODE AND DATASET AVAILABILITY

The source code and experimental materials for this project are available at:

- GitHub repository:
https://github.com/sonbipayacctu10namtrc/GraphShopee_Group2
- Kaggle notebook:
<https://www.kaggle.com/code/sonsosad/graph-t02>

REFERENCES

- [1] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [2] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004.
- [3] Google OR-Tools, "Vehicle routing problem," Google Developers. [Online]. Available: <https://developers.google.com/optimization/routing/vrp>